Assignment 7 — Complete AI Dev Pipeline

ELVTR · Multi-Agent AI for Game Development · Due 3 November 2026, 11:59 PM GMT

Student & game

- **Student:** Soumois Pierre-Emmanuel
- **Game:** Rogue Rigs
- **Concept:** a top-down 3D roguelite auto-shooter built in Unreal Engine 5.8 (C++). An independent miner steals ore from a corporate asteroid claim. The player aims by hand, weapons fire automatically, and the destructible rock arena is the main weapon. A run is 4 levels; upgrades carry over. No AI runs inside the game: the agents are development tools only.

Deliverable 1 — Playable link

- **Playable game:** https://rafale.itch.io/rogue-rigs
- Windows build hosted on itch.io: download, unzip, run `RogueRigs.exe`. No installer, no setup. (Unreal Engine cannot run in a browser.)
- The build on the page was produced and checked on 2 November 2026.

Deliverable 2 — Pipeline source code & engine integration

- **Pipeline repository:** https://github.com/soumois/RogueRigs (instructor has access)
- **Pipeline run video:** https://youtu.be/eL8zYgFhCMU
- **Target engine:** Unreal Engine 5.8 (C++ first, thin Blueprint layer)

Automated flow

- The owner types `/task <request>`. A Planner agent checks it against the GDD and writes task nodes in the vault (`vault/reference/tasks/`). One fresh Executor per task delegates each subtask to a specialised agent, then commits and records its cost.
- **Level-Design agent** writes `Content/RogueRigs/Data/Levels/*.json`; `Tools/validate_layout.js` rejects invalid layouts; the C++ `ARRArenaGrid` loads the file at level start, with no reformatting.
- **Text agent** writes `Content/RogueRigs/Data/Text/*.json` (barks, alerts, upgrade lines). The **Style-Guide agent** scores each line and the refiner rewrites the failing ones (Assignment 6 loop). `Tools/content/validate_text.js` gates the files, then the C++ `RRTextContentSubsystem` loads and displays them in game.
- **C++ agent** writes `Source/RogueRigs/`, built from the command line and checked by 185 headless automation tests.
- **BP agent** creates Blueprints, materials and maps through the Unreal MCP server. **Art** and **3D** agents turn prompts into images and meshes (Meshy, headless Blender) imported through the same MCP.

Deliverable 3 — Pipeline audit & cost analysis

Rogue Rigs · Soumois Pierre-Emmanuel

Pipeline production & functionality

Produced, present in the playable build: 5 arena layouts (JSON, script-validated); 69 player-facing lines (34 barks, 15 alerts, 13 upgrade lines, 5 screen texts, 2 weapon texts) through the style-guide loop; the C++ gameplay (destructible grid, flow field, GAS weapons, 5 robot types, wave director, XP and rarities, shuttle extraction, shop); Blueprints, UMG widgets and Niagara effects; meshes for 8+ robots and props through the art → 3D chain.

Manual steps left: (1) I validate each generated image before it goes to 3D; (2) visual checks and playtests; (3) zipping the packaged build and uploading it to itch.io. Not manual: building, cooking and packaging the game, which Claude Code does when I ask in a direct conversation (outside the crew workflow: a Planner and agents would cost more than a one-step task with no design question), and the editor switching: C++ builds need the editor closed and Blueprint work needs it open, and the agents close and reopen it themselves when a subtask requires it. This only forces editor work to run one subtask at a time on one machine.

To reach 100% automation: a packaging script or agent that builds, zips and runs `butler push` to itch.io from one command, instead of a direct request and a manual upload; automated screenshot comparison instead of my visual review; an image-quality evaluator (the style-guide loop applied to images) to replace my image approval.

Architectural reflection

Decision to change: one long-lived Orchestrator session supervised everything. The first main session ran 335 calls with 22 resumes and peaked at 779k tokens of context, re-reading 137 M cache tokens. **Alternative:** a stateless Executor per global task, fed only by its task node, with state kept in vault files and a short handover note, so context stays under about 100k tokens per run. The project moved this way from 26 October 2026.

Cost analysis

Total actual run cost: about $1,600 at API list prices (3 Oct → 2 Nov 2026), from the project cost log: per-subtask token counts weighted into input-token equivalents (cache write ×1.25, cache read ×0.1, output ×5) and priced at $4 per million input tokens for Opus 5.5 and $2 for Sonnet 5.5. **Actual cash spent: about $70** (two 20 EUR Claude subscriptions for one month, about 10 EUR Meshy, about 10 EUR ChatGPT; at 1.16 USD/EUR), so the subscriptions made the run about 23 times cheaper than API pricing.

Scope	Units (M)	USD
Opus subtasks (mostly the C++ agent)	206.5	826
Sonnet subtasks (Blueprints, text, docs)	164.4	329
Mixed-model subtasks (priced as Opus, upper bound)	8.3	33
Planning, executors, main sessions	135.1	411
Total	**514.2**	**1,599**

Likely a slight underestimate: transcripts of planners, executors and sessions from before 26 October were deleted, and tasks 000–012 are not counted. Cache write is priced at the 5-minute rate.

Most expensive step: the Opus subtasks, mostly the C++ agent, about $826 (52% of the total). Most expensive single subtask: `task-284-01` (Sonnet, 3 runs, 5.57 M units, about $11).

Sustainability: yes. Including about 43 hours of my own supervision (about 10 h/week over 30 days, an estimate) valued at 37.5 EUR/h (about $1,860), the total is about $3,460 at API prices and about $1,930 on real spending. My estimate, not a measurement, of the same prototype built by one developer: at least 2 months at 40 h/week at 37.5 EUR/h, about 12,000 to 13,000 EUR (about $14,000 to $15,000), excluding 3D assets, and a real freelance rate would be higher. So the pipeline is about 4 times cheaper at API prices and 7 to 8 times on real spending. Hardware is excluded on both sides (amortised over several projects). Agents also keep working while I do other things, and one machine per project lets several Unreal pipelines run in parallel without shared assets.

Mid-project cost-reduction change

Strategy — Before: Blueprint subtasks bundled many assets (`task-014-04`, `016-03`, `022-02`, `024-05`). **After** (rule from 14 October): one main asset per Blueprint subtask.

Cost — Before: 4 subtasks, average 2.46 M units (about $4.93) and 4.5 runs each. **After:** 57 subtasks (task 203 onward), average 0.28 M units (about $0.57) and 1.02 runs each. Caveat: these are per-subtask averages, and the bundled subtasks each covered more than one asset.